

CortaAi — Documentação Técnica Completa

Versão: 1.4 | **Data:** 10/06/2026
Branch: feature/migracao-microservicos
Ambiente produção: ZimaOS 10.147.19.1 | **Domínio:** https://cortaaai.shop

Sumário

- 1. [O que é o CortaAi](#)
- 2. [Por que microsserviços — vantagens e desvantagens](#)
- 3. [Mapa de serviços e responsabilidades](#)
- 4. [Infraestrutura e containerização](#)
- 5. [Autenticação e autorização — como funciona](#)
- 6. [Comunicação entre serviços](#)
- 7. [Features detalhadas — fluxo completo de cada uma](#)
 - 7.1 Cadastro e Login
 - 7.2 Gestão de Barbearia
 - 7.3 Equipe de Barbeiros
 - 7.4 Agendamento
 - 7.5 Pagamentos (Mercado Pago)
 - 7.6 Estoque e Produtos
 - 7.7 Notificações
 - 7.8 Assistente Gustave (IA)
- 8. [Criptografia e LGPD](#)
- 9. [Diagrama geral de comunicação](#)
- 10. [Limites de recurso em produção](#)
- 11. [Lacunas identificadas e próximos incrementos](#)
- 12. [Perguntas prováveis da banca \(com linha de defesa\)](#)

1. O que é o CortaAi

CortaAi é um **marketplace SaaS multi-tenant para barbearias**. Conecta três perfis de usuário:

Perfil	Role	Acesso
Cliente	ROLE_CUSTOMER	Agenda serviços, paga, avalia, favorita barbearias
Barbeiro	ROLE_BARBER + isOwner=false	Gerencia agenda, vê comissões, controla indisponibilidades
Dono (Owner)	ROLE_BARBER + isOwner=true	Tudo do barbeiro + gestão da barbearia, equipe, estoque, financeiro

Analogia: É como se o iFood fosse apenas para barbearias. O dono é o restaurante, o barbeiro é o garçom/cozinheiro, o cliente é o consumidor. Cada barbearia é um tenant isolado — os dados de uma nunca vazam para outra.

2. Por que microsserviços

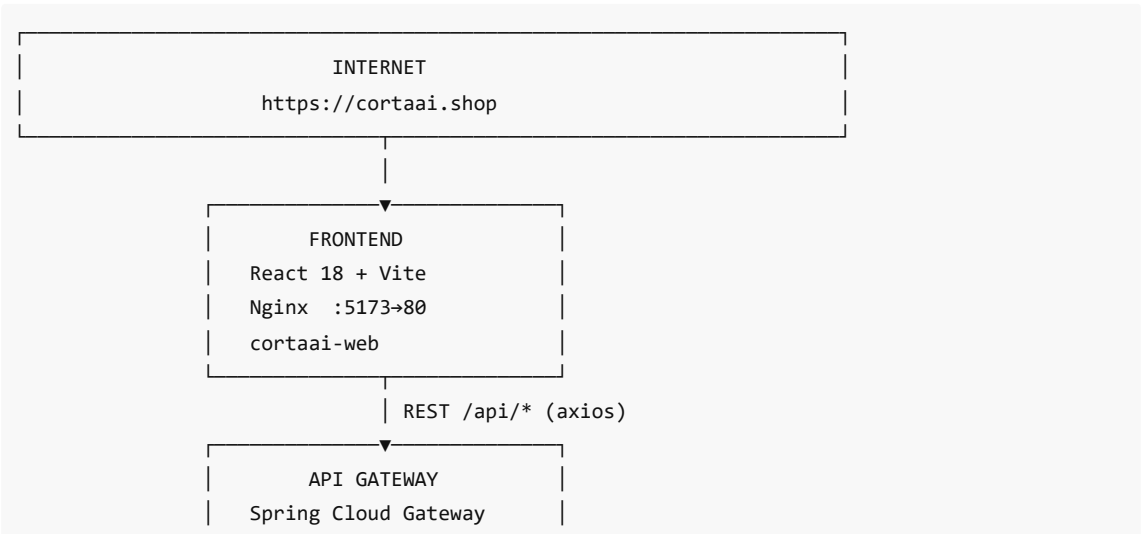
Vantagens

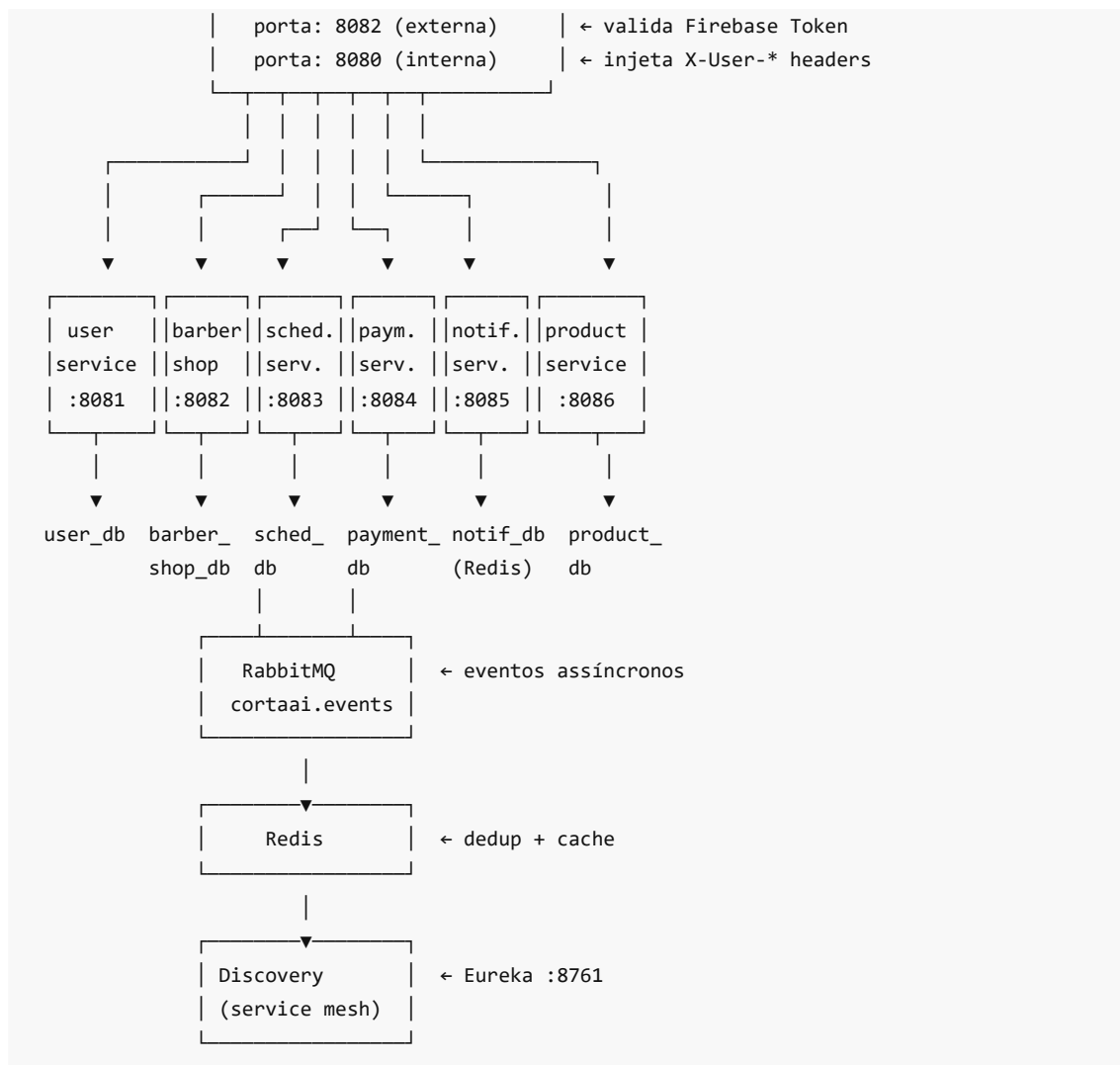
Vantagem	Como se aplica no CortaAi
Escalabilidade independente	Se o agendamento estiver sobrecarregado, sobe só mais uma réplica do <code>schedule-service</code> sem mexer nos outros
Falha isolada	O <code>payment-service</code> cair não derruba o agendamento nem o estoque
Deploy independente	Foi possível corrigir o <code>api-gateway</code> sem redepoyar os outros 6 serviços
Banco isolado por domínio	<code>schedule_db</code> nunca tem JOIN com <code>user_db</code> — dados de clientes não aparecem em queries de pagamento
Times paralelos	Cada serviço pode ser desenvolvido por uma equipe independente

Desvantagens (reais, sentidas no projeto)

Desvantagem	Exemplo concreto no CortaAi
Complexidade operacional	Precisou de Eureka + Gateway + Docker Compose com healthchecks encadeados — 12 containers para subir
Latência extra	Ao criar um agendamento, o <code>schedule-service</code> faz 2 chamadas Feign (<code>user</code> + <code>barbershop</code>) antes de salvar
Transações distribuídas	Não existe <code>@Transactional</code> entre serviços — se o <code>payment-service</code> cair após o agendamento ser criado, precisa de saga/compensação
Debugging difícil	Um erro num agendamento pode envolver logs em 4 containers diferentes
Consumo de memória	8 JVMs rodando simultaneamente — mínimo ~2.5GB RAM só de Java
Cold start	Primeiro request após subir demora porque Eureka ainda não propagou o registro

3. Mapa de Serviços





Responsabilidades

Serviço	Dono dos dados	Faz Feign para	Publica eventos
user-service	barbers, customers, barber_work_blocks	—	—
barbershop-service	barbershops, activities, join_requests	user-service	barbershop.join-request.created
schedule-service	appointments, barber_blocks	user-service, barbershop-service	appointment.*
payment-service	transactions, webhook_logs, dashboard_kpi_daily	user-service, schedule-service, barbershop-service, product-service	payment.approved
notification-service	notifications, device_tokens	—	consome tudo

product-service	products, stock_movements, categories	—	—
-----------------	--	---	---

4. Infraestrutura

Containerização

Todos os serviços rodam em containers Docker gerenciados pelo **Docker Compose**. Em produção usa `docker-compose.server.yml` com `.env.prod`.

Servidor ZimaOS (10.147.19.1)		
└─ cortaa-mysql	MySQL 8.0	porta host: 3307
	└─ user_db	
	└─ barbershop_db	
	└─ schedule_db	
	└─ payment_db	
	└─ product_db	
	└─ notification_db	
└─ cortaa-rabbitmq	RabbitMQ 3	porta host: 5673 (AMQP)
		porta host: 15673 (Management UI)
└─ cortaa-redis	Redis 7	porta host: 6380
└─ discovery-service	Eureka	porta host: 8761
└─ api-gateway	Spring Gateway	porta host: 8082 ← só este é público
└─ user-service		sem porta pública
└─ barbershop-service		sem porta pública
└─ schedule-service		sem porta pública
└─ payment-service		sem porta pública
└─ notification-service		sem porta pública
└─ product-service		sem porta pública
└─ cortaa-web	Nginx + React	porta host: 5173

Analogia: É como um condomínio. O portão de entrada é o `api-gateway` na porta 8082. Os apartamentos (microserviços) ficam atrás do portão e não têm campainha pública — só se comunicam internamente pela rede Docker `cortaa-network`.

Ordem de inicialização (healthchecks encadeados)

MySQL (healthy)
└─ discovery-service (healthy)
└─ api-gateway
└─ user-service
└─ barbershop-service ← RabbitMQ (healthy)
└─ schedule-service ← RabbitMQ (healthy) + Redis (healthy)
└─ payment-service ← RabbitMQ (healthy)
└─ notification-service ← RabbitMQ (healthy)
└─ product-service ← RabbitMQ (healthy)

frontend ← gateway (qualquer estado)

Limites de recurso (produção)

Container	CPU	RAM
MySQL	1 core	384 MB
RabbitMQ	1 core	300 MB
Redis	1 core	48 MB
discovery-service	1 core	256 MB
api-gateway	1 core	300 MB
user-service	1 core	384 MB
barbershop-service	1 core	384 MB
schedule-service	1 core	384 MB
payment-service	1 core	384 MB
notification-service	1 core	384 MB
product-service	1 core	320 MB
frontend	1 core	32 MB
TOTAL	—	≈ 3.6 GB

5. Autenticação e Autorização

Visão geral

O CortaAi **não gerencia senhas**. Toda identidade é delegada ao **Firebase Authentication** (Google). O sistema usa o Firebase ID Token como prova de identidade.

AUTENTICAÇÃO (quem é você?)

```
[Browser/App]
|
| 1→ Firebase SDK → signInWithEmailAndPassword()
|    ↳ retorna: Firebase ID Token (JWT, expira em 1h)
|
| 2→ POST /api/auth/verify { idToken, userType }
|    ↳ user-service valida token com Firebase Admin SDK
|      ↳ se novo usuário: cria registro no banco
|      ↳ retorna: { profileComplete, id, role, isOwner }
```

AUTORIZAÇÃO (o que você pode fazer?)

Todas as requisições protegidas:

Authorization: Bearer <Firebase ID Token>

|

▼

[api-gateway :8082]

└─ verifica assinatura do token (RSA, chaves públicas Google)

└─ extrai custom claims: { role, isOwner }

└─ injeta headers downstream:

X-User-Id = Firebase UID (ex: "59GSw6dVmiMn6rN4Lp8571zTsv13")

X-User-Email = email do usuário

X-User-Role = "ROLE_CUSTOMER" ou "ROLE_BARBER"

X-User-Owner = "true" ou "false"

|

▼

[microserviço destino]

└─ confia nos headers – não valida token novamente

principal.getName() = Firebase UID

Como o campo `isOwner` funciona

O `isOwner` **não é um role separado** — é um **Firestore Custom Claim booleano** que coexiste com `ROLE_BARBER` :

Barbeiro se cadastra → `isOwner = false`

|

Registra barbearia

|

▼

barbershop-service cria Barbershop

|

Feign → `user-service.setCustomUserClaims(uid, "ROLE_BARBER", true)`

|

▼

Firebase atualiza custom claim: { role: "ROLE_BARBER", isOwner: true }

|

Próximo login: token novo já vem com `isOwner = true`

|

api-gateway injeta: `X-User-Owner: "true"`

Rotas públicas (sem token)

<code>/api/auth/verify</code>	← provisionamento inicial
<code>/api/auth/email/**</code>	← login, registro, recuperação de senha
<code>/api/payments/webhook</code>	← webhook Mercado Pago (sem auth)
<code>/api/payments/mp-callback</code>	← OAuth redirect MP
<code>/api/barbershops/{id}</code>	← perfil público de barbearia
<code>/api/barbershops/{id}/barbers</code>	← lista de barbeiros (público)
<code>/actuator/**</code>	← health checks

6. Comunicação entre Serviços

O CortaAi usa **dois padrões complementares** de comunicação inter-serviço:

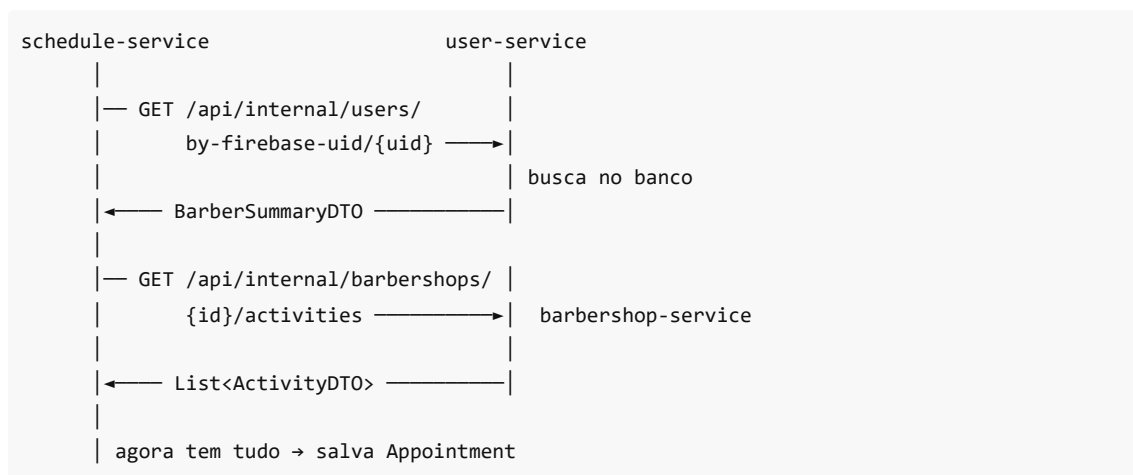
6.1 Síncrona — OpenFeign (consultas)

Analogia: É como ligar para alguém e esperar a resposta. Você fica bloqueado até obter a resposta ou o tempo esgotar.

Quando usar: quando precisa da resposta para continuar (ex: validar se o barbeiro pertence à barbearia antes de criar o agendamento).

Características:

- Descoberta via Eureka: `lb://user-service` → resolve o IP real dinamicamente
- Timeout configurado (evita travamento indefinido)
- `fallback` obrigatório: se o serviço chamado cair, retorna um valor seguro em vez de propagar exceção
- Prefixo `/api/internal/` — não atravessa o Gateway (comunicação direta na rede Docker)

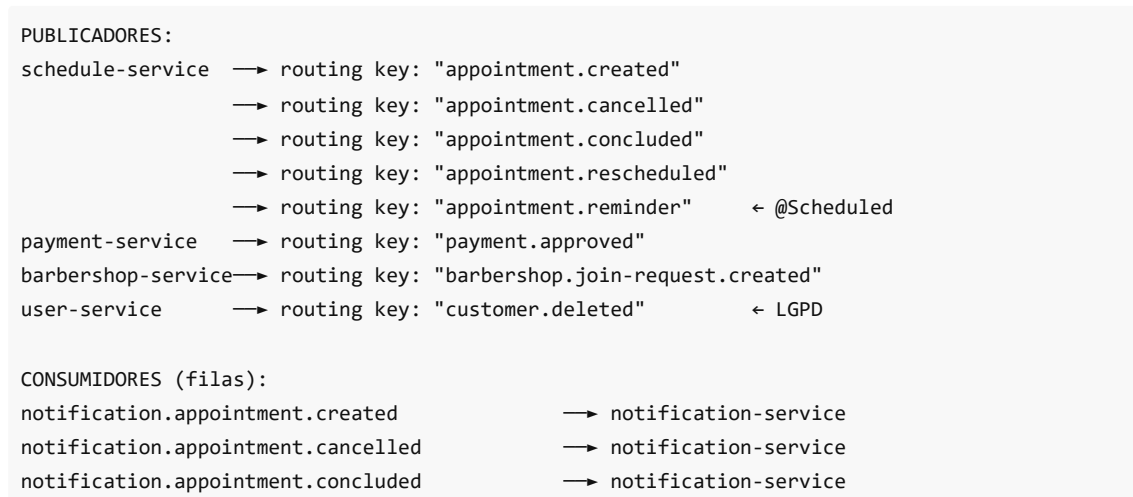


6.2 Assíncrona — RabbitMQ (eventos de domínio)

Analogia: É como mandar uma carta. Você escreve, coloca na caixa do correio e continua sua vida. O destinatário lê quando puder — você não precisa esperar.

Quando usar: para comunicar **mudanças de estado** que outros serviços precisam reagir (notificações, atualização de status, LGPD).

Exchange único: `cortaaai.events` (TopicExchange, durable)



notification.appointment.rescheduled	→	notification-service	
notification.appointment.reminder	→	notification-service	
notification.payment.approved	→	notification-service	
notification.barbershop.join-request.created	→	notification-service	
notification.barber.removed	→	notification-service	
schedule.customer.deleted	→	schedule-service	← anonimiza agendamentos
payment.customer.deleted	→	payment-service	← anonimiza transações
notification.customer.deleted	→	notification-service	

Garantias:

- Mensagens são `durable` — sobrevivem a restart do RabbitMQ
- DLQ (Dead Letter Queue) para eventos críticos
- Idempotência: consumidor usa Redis para verificar se já processou o evento

6.3 Cache — Redis

Usado em dois contextos distintos:

Uso	Serviço	O que guarda
Deduplicação de notificações	notification-service	ID do evento processado (TTL curto) — evita enviar 2x o mesmo e-mail
Deduplicação de agendamentos	schedule-service	Chave de idempotência do request (evita agendamento duplicado por double-click)
Rate limiting	api-gateway	Contador de requests por IP/usuário
Sessão (opcional)	api-gateway	Cookie de sessão (feature flag <code>session.cookie.enabled</code>)

7. Features Detalhadas

7.1 Cadastro e Login

Dois perfis de cadastro: Cliente e Barbeiro.

FLUXO CLIENTE:

1. Firebase SDK → `createUserWithEmailAndPassword(email, senha)`
↳ retorna: Firebase ID Token
2. POST `/api/auth/verify`
body: `{ idToken, userType: "CUSTOMER" }`
↳ user-service cria registro em `customers`
↳ retorna: `{ profileComplete: false, id: null }`
3. POST `/api/auth/customers/complete-profile` [Authorization: Bearer token]
body: `{ name, tell, documentCPF, birthDate }`
↳ user-service:
├─ criptografa CPF, email, telefone com AES/GCM
├─ grava `email_hash` (SHA-256) para busca futura
↳ retorna: `CustomerResponseDTO`

4. Próximas requisições: Authorization: Bearer <token>
└ api-gateway valida, injeta X-User-Id, X-User-Role: ROLE_CUSTOMER

FLUXO BARBEIRO OWNER:

Passos 1-3 idênticos (userType: "BARBER")

4. POST /api/barbershops [Authorization: Bearer token]
body: { name, cnpj, address, latitude?, longitude? }
└ barbershop-service:
└─ criptografa CNPJ
└─ cria Barbershop com owner_id = X-User-Id
└─ se latitude/longitude omitidos → GeocodingService.geocode(address)
│ via Nominatim (OpenStreetMap), 3 tentativas em cascata
│ (endereço completo → rua+cidade → só CEP)
└─ Feign → user-service: atualiza barbers.barbershop_id
└─ Firebase: setCustomUserClaims(uid, "ROLE_BARBER", isOwner=true)
5. Próximo token renovado: claim isOwner = true
└ api-gateway injeta X-User-Owner: "true"
└ frontend libera Dashboard, Estoque, Equipe, Serviços

Login social (Google):

Firebase SDK → signInWithPopup(GoogleAuthProvider)
└ retorna: idToken com providerId = "google.com"
POST /api/auth/verify → user-service cria/encontra usuário por firebase_uid
└ authProvider = "GOOGLE" gravado no banco

7.2 Gestão de Barbearia

Quem pode: ROLE_BARBER + isOwner=true (validado via X-User-Owner: true)

CADASTRO DA BARBEARIA:

POST /api/barbershops
body: { name, cnpj, address, latitude?, longitude? }
└ latitude/longitude são opcionais:
└─ se omitidos → geocodificação automática via Nominatim (OpenStreetMap)
│ 3 tentativas: endereço completo → rua+cidade → CEP
│ falha silenciosa (não bloqueia o cadastro)
└─ se fornecidos → override manual (prevalece sobre geocodificação)
└─ atualiza Firebase custom claim: isOwner = true

UPLOAD DE IMAGENS (logo, banner, portfólio):

POST /api/barbershops/{id}/logo
POST /api/barbershops/{id}/banner
POST /api/barbershops/{id}/highlights
└ arquivo enviado como multipart/form-data
└ barbershop-service → Cloudinary SDK:
└─ faz upload para CDN Cloudinary
└─ recebe: { url, public_id }

- └ salva: barbershops.logo_url + logo_url_public_id
- └ para deletar imagem: Cloudinary.destroy(public_id)

SERVIÇOS (activities):

POST /api/barbershops/{id}/activities

body: { activityName, price, durationMinutes }
└ cria Activity vinculada à barbearia

HORÁRIO DE FUNCIONAMENTO:

(embutido em barbershop – campo workingHours)

AVALIAÇÕES (público):

GET /api/barbershops/{id}/reviews

POST /api/barbershops/{id}/reviews [ROLE_CUSTOMER]

body: { rating: 4, comment: "Ótimo atendimento" }
└ constraint UNIQUE: (customer_id, barbershop_id)
└ um cliente avalia cada barbearia no máximo 1 vez

DESPESAS FIXAS:

POST /api/barbershops/{id}/fixed-expenses

body: { category: "ALUGUEL", amount: 2500.00, month: 5, year: 2026 }
└ categoria enum: AGUA, LUZ, ALUGUEL, INTERNET, ENERGIA,
FUNCIONARIOS, MATERIAL, SISTEMA, CONTABILIDADE,
MARKETING, MANUTENCAO, OUTROS

REGRAS DE COMISSÃO:

POST /api/barbershops/{id}/commission-rules

body: { barberId, activityId, percentage: 40.00 }
└ define que barbeiro X recebe 40% por serviço Y
└ usado no split do Mercado Pago

7.3 Equipe de Barbeiros

Dois fluxos de vinculação:

FLUXO JOIN (barbeiro solicita):

Barbeiro:

POST /api/barbershops/join-request
body: { barbershopId }
└ cria BarbershopJoinRequest(type=JOIN, status=PENDING)
└ publica evento: barbershop.join-request.created
└ notification-service envia notificação ao owner

Owner:

GET /api/barbershops/my-shop/join-requests
PATCH /api/barbershops/join-requests/{id}/approve
└ barbershop-service:
└ status → APPROVED
└ Feign → user-service: barbers.barbershop_id = barbershopId

FLUXO INVITE (owner convida):

Owner:

POST /api/barbershops/{shopId}/invite
body: { barberEmail }
└─ cria BarbershopJoinRequest(type=INVITE, status=PENDING)
└─ publica evento: barbershop.join-request.created
└─ notification-service envia convite ao barbeiro

Barbeiro:

GET /api/barbershops/my-invites
POST /api/barbershops/accept-invite/{inviteId}
POST /api/barbershops/reject-invite/{inviteId}

REMOÇÃO:

Owner:

DELETE /api/barbershops/{shopId}/barbers/{barberId}
└─ Feign → user-service: barbers.barbershop_id = null
└─ publica evento: barber.removed

HABILIDADES DO BARBEIRO:

PATCH /api/barbers/{id}/activities
body: { activityIds: ["uuid1", "uuid2"] }
└─ atualiza barber_assigned_activities
└─ define quais serviços o barbeiro realiza
└─ só estes serviços aparecem na tela de agendamento para o cliente

7.4 Agendamento

O coração do negócio. É o serviço mais complexo — faz Feign para 2 serviços antes de salvar.

VERIFICAR DISPONIBILIDADE:

GET /api/appointments/availability
params: { barberId, date, activityIds[] }
└─ schedule-service:
└─ Feign → user-service: busca barber_work_blocks (horários do dia)
└─ Feign → barbershop-service: busca duração de cada serviço
└─ busca appointments existentes do barbeiro naquele dia
└─ busca barber_blocks (bloqueios manuais)
└─ retorna: lista de slots livres (hora a hora)

CRIAR AGENDAMENTO:

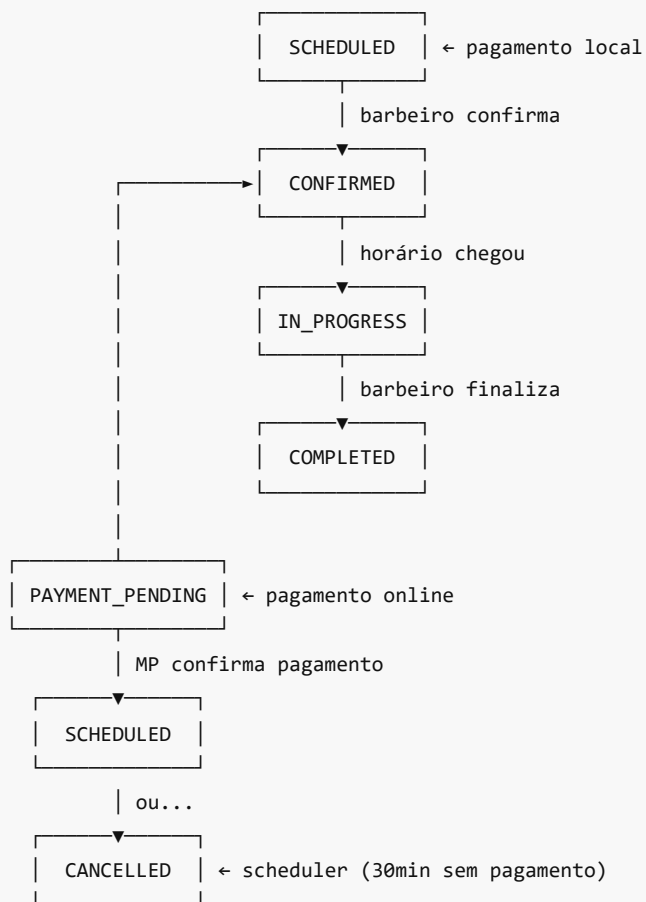
POST /api/appointments
body: {
 barberId, barbershopId,
 activityIds: ["uuid1", "uuid2"],
 startTime: "2026-05-24T10:00:00",
 paymentMethod: "PIX" // ou "LOCAL"
}
└─ schedule-service:
└─ Feign → user-service: valida barbeiro + snapshot do nome
└─ Feign → barbershop-service: valida serviços + snapshot de nome/preço/duração
└─ calcula: end_time = start_time + soma(durationMinutes)

```

└─ calcula: total_price = soma(price)
└─ verifica conflito: nenhum appointment ativo no intervalo para o barbeiro
└─ se paymentMethod = PIX/CARTÃO:
    └─ status = PAYMENT_PENDING
    └─ aguarda payment-service gerar checkout
└─ se paymentMethod = LOCAL:
    └─ status = SCHEDULED
└─ publica: appointment.created
    └─ notification-service notifica barbeiro

```

CICLO DE VIDA DO AGENDAMENTO:



STATUS ESPECIAIS:

WALK_IN → atendimento imediato, sem agendamento prévio
 CANCELLED → cancelado por cliente ou barbeiro
 NO_SHOW → cliente não compareceu
 EXPIRED → projeção lazy (PAYMENT_PENDING + start_time passado)

SCHEDULER AUTOMÁTICO (a cada 5 minutos):

```

AppointmentLifecycleScheduler:
└─ cancelExpiredPayments:
    └─ appointments com status=PAYMENT_PENDING
        e date_created + 30min < now
        → status = CANCELLED
└─ autoCompleteAppointments:

```

```
└─ appointments com status=IN_PROGRESS
   e end_time < now
   → status = COMPLETED
   → publica: appointment.concluded
```

SCHEDULER DE LEMBRETE (a cada 5 minutos):

```
ReminderScheduler:
├─ timezone padrão de execução: UTC (configurável por `app.timezone`)
├─ consulta janela: `now` até `now + 1 hora`
├─ publica: appointment.reminder para appointments no recorte
└─ não dispara lembrete antes da janela de 1 hora
```

WALK-IN:

POST /api/appointments/walk-in

```
body: { barberId, barbershopId, activityIds[] }
└─ start_time = now(), sem verificação de conflito antecipado
└─ status = WALK_IN
```

BLOQUEIO DE AGENDA (barbeiro):

POST /api/barber-blocks

```
body: { startTime, endTime, reason: "Almoço" }
└─ impede novos agendamentos no período
```

7.5 Pagamentos (Mercado Pago)

Split marketplace: o pagamento é dividido automaticamente entre a plataforma CortaAi e o barbeiro.

FLUXO COMPLETO:

1. Cliente cria agendamento com paymentMethod = "PIX"
 - └─ schedule-service publica: appointment.created
2. payment-service consome appointment.created
 - └─ cria Transaction(status=PENDING, amount=R\$50,00)
 - └─ chama API Mercado Pago:

```
POST /checkout/preferences
body: {
  items: [{ title, price }],
  marketplace_fee: R$5,00, ← taxa CortaAi
  collector_id: mpUserId do barbeiro (para split)
}
```
 - └─ MP retorna: { preferenceId, checkoutUrl }
 - └─ Transaction.checkoutUrl = "https://www.mercadopago.com.br/checkout/..."
 - └─ Feign → schedule-service: atualiza appointment com checkoutUrl
3. Cliente acessa checkoutUrl → paga no ambiente MP
 - └─ MP processa o pagamento
4. MP envia webhook para /api/payments/webhook (rota PÚBLICA)
 - └─ payment-service:
 - ├─ verifica idempotência: webhook_logs.mp_resource_id (UNIQUE)
 - ├─ se já processado → ignora

```
└─ busca detalhes do pagamento na API MP
└─ atualiza Transaction:
  │   └─ status = APPROVED
  │   └─ gross_amount = R$50,00 ← pago pelo cliente
  │   └─ mp_fee_amount = R$2,50 ← taxa MP (~5%)
  │   └─ platform_fee_amount = R$5,00 ← taxa CortaAi
  │   └─ net_amount = R$42,50 ← repasse ao barbeiro
└─ atualiza dashboard_kpi_daily (UPSERT por barbershop_id + data)
  └─ publica: payment.approved
```

5. notification-service consome payment.approved
└─ envia notificação ao cliente: "Pagamento confirmado!"

6. schedule-service consome payment.approved (via Feign)
└─ appointment.status = SCHEDULED

OAUTH DO LOJISTA (para split):

GET /api/payments/connect-mercadopago [ROLE_BARBER + isOwner]
└─ redirect para: https://auth.mercadopago.com/authorization?...
└─ owner autoriza o CortaAi a fazer split em nome dele

Callback:

GET /api/payments/mp-callback?code=xxx (rota PÚBLICA)
└─ payment-service troca code por access_token
└─ Feign → user-service: salva mp_access_token no banco (criptografado)
└─ owner.mp_user_id = collector_id no MP

7.6 Estoque e Produtos

Quem pode: ROLE_BARBER + isOwner=true

PRODUTOS:

POST /api/products
body: { name, description, price, categoryId, stockQuantity, minStockQuantity }
└─ cria Product(barbershop_id do owner, active=true)

CATEGORIAS DINÂMICAS:

POST /api/products/categories
body: { name: "Pomadas" }
└─ category.name UNIQUE por barbearia

MOVIMENTAÇÕES DE ESTOQUE:

POST /api/products/{id}/stock-movements
body: { type, quantity, notes }

Tipos disponíveis:

Tipo	Sinal	Quando usar
IN	+	Compra / reposição de estoque
OUT	-	Saída genérica
OUT_CONSUMPTION	-	Produto usado no atendimento

OUT_SALE	-	Produto vendido ao cliente
LOSS	-	Quebra, vencimento, extravio
RETURN	+	Devolução ao fornecedor

- └ atualiza products.stock_quantity
- └ registra histórico imutável em stock_movements

ALERTA DE REPOSIÇÃO:

```
GET /api/products?lowStock=true
└ retorna products onde stock_quantity <= min_stock_quantity
└ view v_stock_health_alert no banco
```

7.7 Notificações

Consumidor puro: só consome eventos RabbitMQ, nunca publica.

CANAIS DISPONÍVEIS:

Canal	Como funciona
IN_APP	Salvo em notification_db, cliente lê via GET
EMAIL	Spring Mail (SMTP), templates Thymeleaf
PUSH (FCM)	Firestore Cloud Messaging, token em device_tokens

EVENTOS QUE DISPARAM NOTIFICAÇÃO:

```
appointment.created      → barbeiro recebe: "Novo agendamento"
appointment.cancelled    → ambos recebem: "Agendamento cancelado"
appointment.concluded    → cliente recebe: "Como foi? Avalie!"
appointment.rescheduled → ambos recebem: "Agendamento remarcado"
appointment.reminder     → cliente recebe: "Lembrete: seu horário está próximo"
payment.approved         → cliente recebe: "Pagamento confirmado"
join-request.created     → owner recebe: "Barbeiro solicitou entrar na equipe"
                        → barbeiro recebe: "Você recebeu um convite"
```

TIPOS PERSISTIDOS EM `notification\_db.notifications.type`:

```
APPOINTMENT_CREATED
APPOINTMENT_CANCELLED
APPOINTMENT_CONCLUDED
APPOINTMENT_RESCHEDULED
APPOINTMENT_REMINDER
PAYMENT_APPROVED
BARBERSHOP_JOIN_REQUEST_CREATED
BARBER_REMOVED
```

IDEMPOTÊNCIA:

```
notification-service → Redis.get("notif:{eventId}")
└ se existe: ignora (já enviou)
└ se não existe:
  └ processa notificação
    └ Redis.setex("notif:{eventId}", 3600, "processed")
```

```
REGISTRO DE DEVICE TOKEN (push):
POST /api/notifications/device-tokens
  body: { token: "fcm-token...", platform: "WEB" }
    ↳ salva em device_tokens com constraint UNIQUE no token
    ↳ plataforma suportada atualmente: WEB (via Firebase Messaging SW)
```

7.8 Assistente Gustave (IA)

Assistente conversacional integrado ao `schedule-service`. Usa múltiplos provedores com fallback automático.

PROVEDORES (4 configurados, ativados via variável de ambiente):

1. Gemini – gemini-2.0-flash (GEMINI_API_KEY)
2. Groq – llama-3.3-70b-versatile (GROQ_API_KEY + GROQ_MODEL)
3. OpenRouter – meta-llama/llama-3.1-8b-instruct:free (OPENROUTER_API_KEY + OPENROUTER_MODEL)
4. Cohere – command-a-03-2025 (COHERE_API_KEY + COHERE_MODEL)

A ordem de tentativa e quais provedores estão ativos depende das chaves configuradas no `.env.prod`. A lógica de fallback percorre os provedores em ordem até obter resposta.

FLUXO:

```
POST /api/schedule/ai/chat [Authorization: Bearer token]
  body: { message: "Quero agendar um corte amanhã às 10h" }
    ↳ schedule-service tenta provedores configurados em sequência
    ↳ resposta em linguagem natural
    ↳ pode sugerir horários disponíveis com base na agenda real
```

DELETE /api/schedule/ai/chat/history ← limpa histórico do usuário (Redis)

CONTEXTO DISPONÍVEL PARA A IA:

- ↳ agenda do barbeiro (via banco local)
- ↳ serviços da barbearia (via Feign → barbershop-service)
- ↳ horários disponíveis (cálculo interno)
- ↳ histórico da conversa armazenado no Redis (ChatHistoryService)

8. Criptografia e LGPD

Dados criptografados em repouso (AES/GCM)

Chave: variável de ambiente `CORTAAI_DATA_CRYPT0_KEY`

Campos criptografados com `SensitiveStringConverter` (@Converter JPA):

Tabela	Campo	Tipo de dado
barbers	email	e-mail do barbeiro
barbers	tell	telefone
barbers	document_cpf	CPF
barbers	birth_date	data de nascimento

barbers	mp_refresh_token	OAuth token MP
barbers	mp_user_id	collector ID MP
customers	email	e-mail do cliente
customers	tell	telefone
customers	document_cpf	CPF
customers	birth_date	data de nascimento
barbershops	cnpj	CNPJ da barbearia
appointments	customer_name	nome no agendamento

```
email_hash = SHA-256(email antes de criptografar)
└─ permite buscar por e-mail sem descriptografar:
    SELECT * FROM barbers WHERE email_hash = SHA2('email@test.com', 256)
```

Anonimização (LGPD — art. 18)

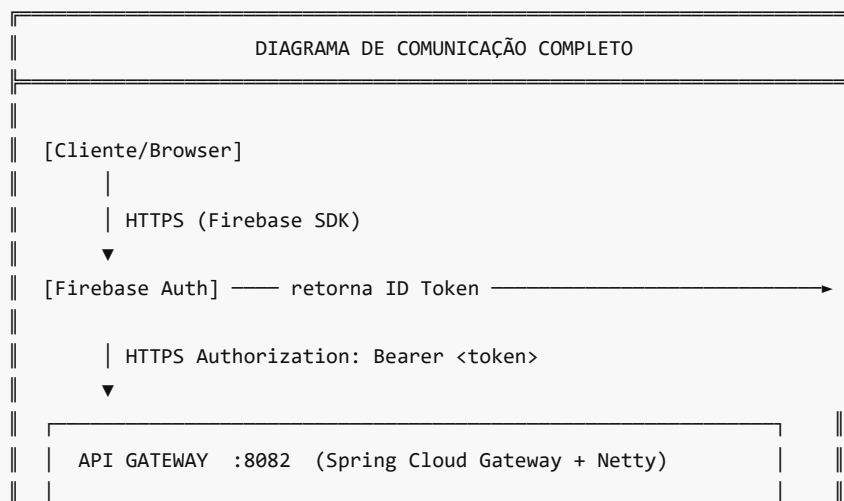
```
Cliente solicita exclusão de conta:
DELETE /api/customers/me [ROLE_CUSTOMER]
└─ user-service:
    └─ deleta registro de customers
        └─ publica: customer.deleted

schedule-service consome customer.deleted:
└─ appointments.customer_name = "Cliente Removido" (anonimiza)
└─ appointments futuros → status = CANCELLED

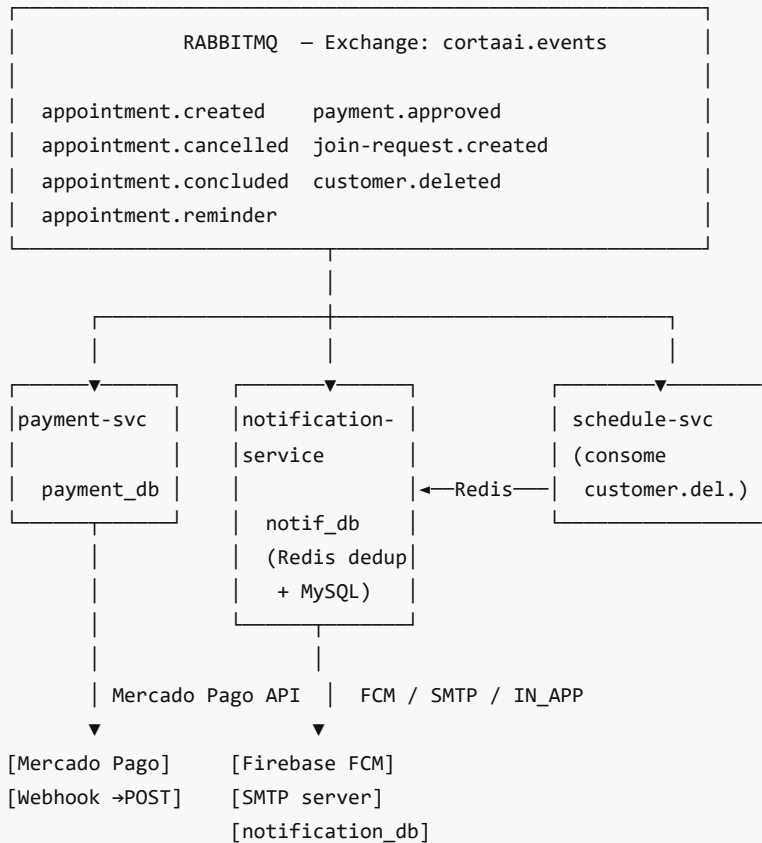
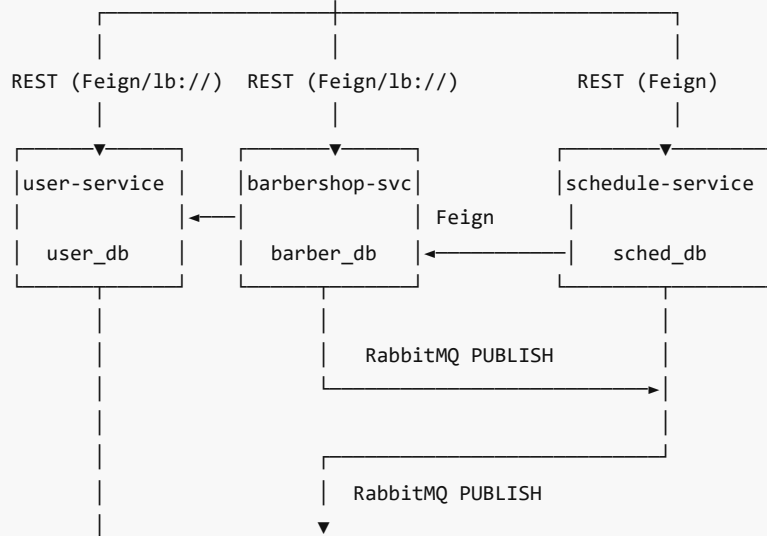
payment-service consome customer.deleted:
└─ transactions.customer_id = null (anonimiza)

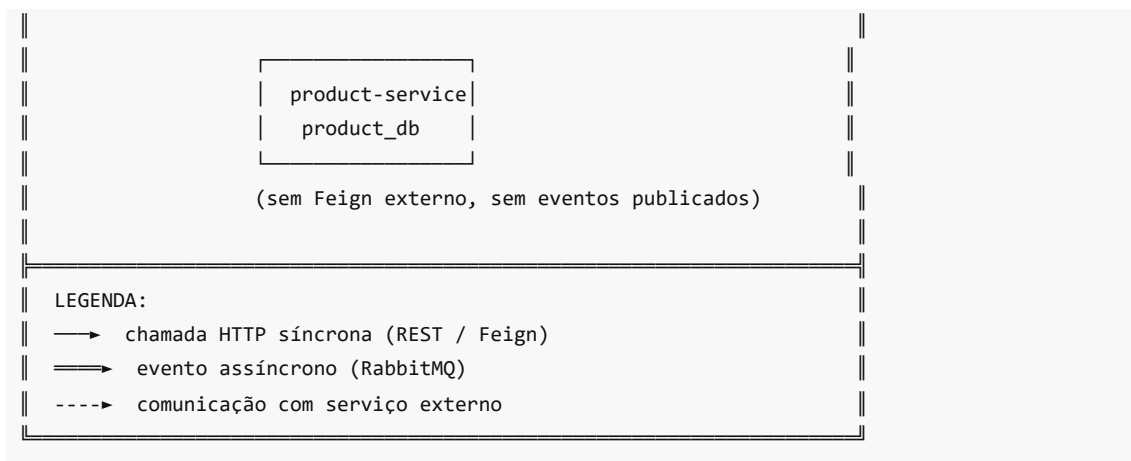
notification-service consome customer.deleted:
└─ deleta notificações do usuário
```

9. Diagrama Geral de Comunicação



1. Valida token (RSA públicas Google)
2. Extraí claims: role, isOwner
3. Injeta headers:
 - X-User-Id → Firebase UID
 - X-User-Email → email
 - X-User-Role → ROLE_CUSTOMER | ROLE_BARBER
 - X-User-Owner → true | false
4. Rate limiting via Redis
5. Roteia via Eureka (lb://servico-name)





10. Limites de Recurso em Produção

O servidor ZimaOS é uma máquina de uso doméstico/SMB. Os limites foram ajustados para caber dentro da RAM disponível:

Redis:

```
maxmemory 32mb  
maxmemory-policy allkeys-lru ← remove chaves menos usadas quando cheio
```

MySQL:

```
innodb-buffer-pool-size = 128M  
max-connections = 50  
performance-schema = OFF ← economiza ~30MB
```

JVMs (todos os serviços Java):

```
-XX:+UseContainerSupport  
-XX:MaxRAMPercentage=65.0 ← usa 65% do limite do container  
-XX:MetaspaceSize=64m  
-XX:MaxMetaspaceSize=96m
```

Portas externas (diferentes do padrão para não conflitar com ZimaOS):

```
MySQL: 3307 (padrão: 3306)  
RabbitMQ: 5673 (padrão: 5672)  
Redis: 6380 (padrão: 6379)  
Gateway: 8082 (interno: 8080)  
Frontend: 5173 (porta Vite – nginx interno serve na 80)
```

11. Lacunas Identificadas e Próximos Incrementos

Apesar da arquitetura estar funcional e consistente com o modelo de microsserviços, estes pontos podem ser cobrados pela banca como "dívida técnica controlada":

11.1 Observabilidade ponta a ponta

Estado atual: logs por serviço e troubleshooting manual por container.

Lacuna: ausência de rastreamento distribuído e correlação explícita entre requisições HTTP, Feign e eventos RabbitMQ.

Incremento recomendado: padronizar `correlationId` (header + payload de evento), agregar logs centralizados e adicionar métricas de negócio (ex.: taxa de conversão de `PAYMENT_PENDING` para `SCHEDULED`).

11.2 Resiliência de integração externa

Estado atual: fallback em Feign e idempotência em partes críticas.

Lacuna: política de circuit breaker/retry/backoff não está documentada de forma uniforme para todos os clientes externos (Mercado Pago, provedores de IA).

Incremento recomendado: formalizar matriz por integração com timeout, retries máximos, janelas de cooldown e comportamento degradado esperado.

11.3 Estratégia formal de backup e disaster recovery

Estado atual: ambiente produtivo containerizado com limites de recurso claros.

Lacuna: falta de capítulo consolidando RPO/RTO, rotina de backup dos bancos, teste de restauração e procedimento de rollback operacional.

Incremento recomendado: adicionar runbook de continuidade com frequência de backup, retenção, teste mensal de restore e critérios de failover.

11.4 Segurança operacional além da autenticação

Estado atual: autenticação via Firebase e criptografia AES/GCM para dados sensíveis.

Lacuna: rotação de segredos/chaves e trilha de auditoria de ações administrativas não estão descritas de ponta a ponta.

Incremento recomendado: política de rotação de credenciais (Firebase, MP, chaves criptográficas), auditoria para endpoints críticos e checklist de hardening por ambiente.

11.5 Governança de contratos entre serviços

Estado atual: contratos REST e eventos já definidos na prática.

Lacuna: versionamento formal de eventos e compatibilidade retroativa não estão explicitados como política de evolução.

Incremento recomendado: estabelecer convenção de versionamento (`v1` , `v2`) para DTO/eventos e critérios de depreciação com janela mínima.

11.6 Evidência de qualidade contínua (CI/CD)

Estado atual: cobertura de testes consolidada (JaCoCo).

Lacuna: pipeline de quality gate, testes de contrato e smoke tests pós-deploy não estão descritos no documento principal.

Incremento recomendado: incluir fluxo de CI/CD com etapas obrigatórias (build, testes, análise estática, scan de segurança e deploy controlado).

12. Perguntas Prováveis da Banca (com Linha de Defesa)

12.1 Arquitetura e decisões

1. Por que microsserviços e não monolito modular para este escopo?

Linha de defesa: isolamento por domínio, deploy independente e desacoplamento de falhas já trouxeram ganho real no projeto; a complexidade adicional foi tratada com gateway, discovery e mensageria.

2. Como vocês evitam acoplamento entre bancos se há referências cross-service?

Linha de defesa: cada serviço possui banco próprio; referências são UUIDs lógicos sem FK física entre

bancos; integração ocorre por contrato (Feign/eventos).

3. Quando escolher Feign vs RabbitMQ?

Linha de defesa: consulta que bloqueia fluxo usa Feign; mudança de estado para reação assíncrona usa evento RabbitMQ; regra já aplicada nos fluxos de agendamento, pagamento e notificação.

4. Quais trade-offs de usar Eureka + Gateway em vez de roteamento estático?

Linha de defesa: ganho de descoberta dinâmica e escalabilidade de réplicas; custo de cold start e complexidade operacional foi assumido e documentado.

12.2 Segurança, LGPD e conformidade

5. Como garantem que um serviço interno não seja acessado diretamente?

Linha de defesa: somente gateway exposto externamente; serviços internos ficam na rede privada Docker sem porta pública.

6. Por que confiar em headers `X-User-*` sem validar token em todo serviço?

Linha de defesa: modelo de perímetro controlado no gateway para reduzir custo de validação repetida; tráfego interno restrito e padronizado.

7. Como tratam LGPD no direito ao esquecimento sem quebrar histórico financeiro?

Linha de defesa: anonimização orientada a evento (`customer.deleted`) em serviços consumidores, preservando integridade operacional e removendo vínculo pessoal.

8. Como é feita a proteção de dados sensíveis em repouso e em trânsito?

Linha de defesa: AES/GCM para campos sensíveis e TLS nas comunicações externas; recomendação adicional é formalizar rotação de chaves no runbook.

12.3 Consistência, resiliência e operação

9. Sem transação distribuída, como evitam inconsistência entre agendamento e pagamento?

Linha de defesa: arquitetura orientada a eventos com estados explícitos (`PAYMENT_PENDING` , `SCHEDULED` , `CANCELLED`) e compensações automáticas por scheduler.

10. Como lidam com mensagens duplicadas no RabbitMQ?

Linha de defesa: consumidores idempotentes com Redis para deduplicação por `eventId` e persistência de processamento.

11. Qual o comportamento quando Mercado Pago ou provedor de IA fica indisponível?

Linha de defesa: fallback e tentativa por múltiplos provedores (no caso da IA), além de fluxo degradado; recomendação é evoluir para matriz formal de circuit breaker/retry.

12. Como garantem que o scheduler não produza efeitos indevidos em massa?

Linha de defesa: transições de estado restritas por regra temporal e status atual; operação é determinística e auditável por logs/eventos.

12.4 Escalabilidade e custo

13. Como provar que a solução escala sem aumentar custo linearmente?

Linha de defesa: escalabilidade seletiva por serviço (ex.: `schedule-service`) e limites por container definidos por perfil de carga.

14. Qual serviço é gargalo hoje e qual plano de mitigação?

Linha de defesa: agendamento e pagamento concentram maior criticidade; mitigação envolve cache, tuning

de queries, observabilidade e scaling horizontal dedicado.

15. Com 3.6 GB de RAM total, como evitam degradação em pico?

Linha de defesa: limites de JVM por container, Redis/MySQL ajustados para footprint reduzido e controle de recursos no compose.

12.5 Qualidade de software

16. Cobertura de 85% garante qualidade?

Linha de defesa: cobertura é indicador secundário; qualidade depende de testes de fluxo crítico, integração entre serviços e cenários de falha.

17. Que evidências de não regressão vocês apresentam?

Linha de defesa: suíte automatizada + testes de API dos fluxos críticos; próximo passo é ampliar testes de contrato entre serviços.

18. Como validar compatibilidade quando um evento evolui?

Linha de defesa: manter compatibilidade backward nos consumidores e versionar contratos; ponto já mapeado como incremento formal da governança.

12.6 Produto e negócio

19. Qual risco de dependência de terceiros (Firebase, Mercado Pago, Cloudinary)?

Linha de defesa: dependência consciente com encapsulamento por serviço e fallback operacional; risco residual documentado com plano de contingência.

20. Qual foi o principal ganho de negócio após a migração para microsserviços?

Linha de defesa: maior velocidade de evolução por domínio (deploy e correções independentes) e melhor isolamento de falhas em fluxos críticos.

12.7 Perguntas "de profundidade" que costumam aparecer

21. Mostre um fluxo completo, com request real, passando por 3 serviços e 1 evento.

22. Se o RabbitMQ cair por 10 minutos, o que perde e o que não perde?

23. Se o gateway cair, qual plano de retorno e tempo estimado?

24. Qual decisão técnica vocês tomariam diferente se comessem hoje?

Para essas perguntas, a melhor estratégia é responder com um caso real do sistema (agendamento + pagamento + notificação), citando status, evento e mecanismo de recuperação.

*Documento atualizado em 10/06/2026 com base em análise estática do código-fonte no branch `feature/migracao-microservicos`. Cobertura JaCoCo global: **85%** (533 testes, 3.861/25.938 instruções missed). Atualização desta versão inclui correções do lembrete de 1 hora (timezone UTC e janela operacional), adequação de enum de notificação (`APPOINTMENT_REMINDER`), além das lacunas técnicas priorizadas e roteiro de perguntas prováveis da banca.*